

Output	
Result 1	
16 rows ↺ ↻ 🔍 📄 CSV 📄	
QUERY PLAN ⌵	
1	Sort (cost=139.10..141.60 rows=1000 width=17) (actual time=3.776..3.803 rows=1000 loops=1)
2	Sort Key: cd.fcs
3	Sort Method: quicksort Memory: 63kB
4	→ Hash Join (cost=66.00..89.27 rows=1000 width=17) (actual time=0.388..0.686 rows=1000 loops=1)
5	Hash Cond: (t.contact_details_id = cd.id)
6	→ Hash Join (cost=29.50..50.14 rows=1000 width=11) (actual time=0.154..0.348 rows=1000 loops=1)
7	Hash Cond: (a.id_tenants = t.id_tenants)
8	→ Seq Scan on active_accounts_view a (cost=0.00..18.00 rows=1000 width=11) (actual time=0.012..0.083 rows=1000 loops=1)
9	→ Hash (cost=17.00..17.00 rows=1000 width=12) (actual time=0.134..0.134 rows=1000 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 55kB
11	→ Seq Scan on tenants t (cost=0.00..17.00 rows=1000 width=12) (actual time=0.008..0.073 rows=1000 loops=1)
12	→ Hash (cost=24.00..24.00 rows=1000 width=18) (actual time=0.222..0.222 rows=1000 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 63kB
14	→ Seq Scan on contact_details cd (cost=0.00..24.00 rows=1000 width=18) (actual time=0.012..0.148 rows=1000 loops=1)
15	Planning Time: 4.542 ms
16	Execution Time: 3.873 ms

Sort (cost=139.10..141.60 rows=1000 width=17) (actual time=3.776..3.803 rows=1000 loops=1)

Sort Key: cd.fcs

Sort Method: quicksort Memory: 63kB

-> Hash Join (cost=66.00..89.27 rows=1000 width=17) (actual time=0.388..0.686 rows=1000 loops=1)

Hash Cond: (t.contact_details_id = cd.id)

-> Hash Join (cost=29.50..50.14 rows=1000 width=11) (actual time=0.154..0.348 rows=1000 loops=1)

Hash Cond: (a.id_tenants = t.id_tenants)

-> Seq Scan on active_accounts_view a (cost=0.00..18.00 rows=1000 width=11) (actual time=0.012..0.083 rows=1000 loops=1)

-> Hash (cost=17.00..17.00 rows=1000 width=12) (actual time=0.134..0.134 rows=1000 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 55kB

-> Seq Scan on tenants t (cost=0.00..17.00 rows=1000 width=12) (actual time=0.008..0.073 rows=1000 loops=1)

-> Hash (cost=24.00..24.00 rows=1000 width=18) (actual time=0.222..0.222 rows=1000 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 63kB

-> Seq Scan on contact_details cd (cost=0.00..24.00 rows=1000 width=18) (actual time=0.012..0.148 rows=1000 loops=1)

1. Seq Scan on contact_details

```
-> Seq Scan on contact_details cd (cost=0.00..24.00 rows=1000 width=18) (actual time=0.012..0.148 rows=1000 loops=1)
```

Что происходит: Последовательное сканирование таблицы contact_details.

Выходные данные: Все строки таблицы contact_details (1000 строк).

Стоимость (cost):

0.00: Стоимость начала операции (чтение первой строки).

24.00: Общая стоимость операции.

Каждая строка занимает 18 байт.

2. Hash

```
Hash (cost=24.00..24.00 rows=1000 width=18) (actual time=0.209..0.210 rows=1000 loops=1)  
Buckets: 1024 Batches: 1 Memory Usage: 63kB
```

Создание хэш-таблицы из строк, полученных из contact_details для использования в Hash Join.

Buckets: Хэш-таблица содержит 1024 buckets для хранения строк.

Затрачено 63 KB оперативной памяти для хранения хэшей.

3. Seq Scan on tenants

```
Seq Scan on tenants t (cost=0.00..17.00 rows=1000 width=12) (actual time=0.008..0.069 rows=1000 loops=1)
```

Что происходит: Последовательное сканирование таблицы tenants.

Выходные данные: Все строки таблицы tenants (1000 строк).

Стоимость:

0.00: Начало сканирования.

17.00: Общая стоимость.

Каждая строка занимает 12 байт.

4. Hash

```
Hash (cost=17.00..17.00 rows=1000 width=12) (actual time=0.132..0.133
rows=1000 loops=1)
Buckets: 1024  Batches: 1  Memory Usage: 55kB
```

Что происходит: Создание хэш-таблицы из строк, полученных из `tenants` для использования в следующем Hash Join.

Затрачено 55 KB оперативной памяти.

5. Seq Scan on active_accounts_view (a)

```
Seq Scan on active_accounts_view a (cost=0.00..18.00 rows=1000 width=11)
(actual time=0.012..0.096 rows=1000 loops=1)
```

Что происходит: Последовательное сканирование представления `active_accounts_view`.

Выходные данные: Все строки представления (1000 строк).

Стоимость:

0.00: Начало операции.

18.00: Общая стоимость.

Каждая строка занимает 11 байт.

6. Hash Join

```
Hash Join (cost=29.50..50.14 rows=1000 width=11) (actual time=0.157..0.371
rows=1000 loops=1)
Hash Cond: (a.id_tenants = t.id_tenants)
```

Что происходит: Выполняется соединение строк из `active_accounts_view` и `tenants`.

Используется хэш-таблица, построенная из `tenants`.

Условие соединения: `a.id_tenants = t.id_tenants`.

Выходные данные: 1000 строк.

Стоимость:

29.50: Начало соединения.

50.14: Общая стоимость.

7. Hash Join

```
Hash Join (cost=66.00..89.27 rows=1000 width=17) (actual time=0.379..0.701
rows=1000 loops=1)
Hash Cond: (t.contact_details_id = cd.id)
```

Что происходит: Выполняется соединение строк из tenants и contact_details.

Используется хэш-таблица, построенная из contact_details.

Условие соединения: t.contact_details_id = cd.id.

Выходные данные: 1000 строк.

Стоимость:

66.00: Начало соединения.

89.27: Общая стоимость.

8. Sort

```
Sort (cost=139.10..141.60 rows=1000 width=17) (actual time=3.800..3.832
rows=1000 loops=1)
Sort Key: cd.fcs
Sort Method: quicksort Memory: 63kB
```

Что происходит: Сортировка результата по полю cd.fcs с использованием алгоритма quicksort.

Использовано 63 KB оперативной памяти.

Стоимость:

139.10: Начало сортировки.

141.60: Общая стоимость.

Planning Time: Планирование запроса заняло 4.542 ms

Execution Time: Выполнение запроса заняло 3.873 ms